

PYTHON PROGRAMMING

1) BASIC 1: save as "01basic.py"

```
print("Hello, BonicBot!")
print("I am learning Python!")
print("I am ready for AI RoboX Challenge 2026!")
```

BASIC 2: save as "01basic2.py"

```
name = "BonicBot"
age = 2
height = 0.80
is_robot = True

print(name)
print(age)
print(height)
print(is_robot)
```

2) VARIABLES AND DATA TYPES: save as "02vdt.py"

```
# variables_and_io.py

# Variables and core data types
name = "Aditi"          # str
age = 24                # int
height_m = 1.68         # float
is_student = True       # bool

print(name, age, height_m, is_student)
print(type(name), type(age), type(height_m), type(is_student))

# Taking input from the user
favorite_color = input("What's your favorite color? ")
print(f"{name} likes {favorite_color}.")

# input() always returns a string – even when the user types
# digits
birth_year_text = input("What year were you born? ")
birth_year = int(birth_year_text)  # convert str -> int
current_year = 2026
approx_age = current_year - birth_year

print(f"You're approximately {approx_age} years old.")
```

3) MATH_OPERATIONS: save as "03math.py"

```
# math_operations.py

a = 17
b = 5

print("Addition:", a + b)
print("Subtraction:", a - b)
print("Multiplication:", a * b)
print("True Division:", a / b)
print("Floor Division:", a // b)
print("Modulus (remainder):", a % b)
print("Exponent:", a ** b)

# Operator precedence (PEMDAS: Parentheses, Exponents, Mult/Div, Add/Sub)
result = 2 + 3 * 4 ** 2 - (6 / 2)

print("Precedence example:", result)
```

4) PRINT AND STRINGS:

```
# print_and_strings.py

name = "Rahul"
score = 91.5

# Basic print
print("Result ready.")

# Multiple values, custom separator and end
print("a", "b", "c", sep=" - ", end="!\n")

# f-strings (the modern, recommended approach)
print(f"{name} scored {score:.1f}%")

# .format() method (common in older codebases)
print("{} scored {:.1f}%".format(name, score))

# %-style formatting (legacy, still seen in older Python code)
print("%s scored %.1f%%" % (name, score))

# String operations
greeting = "Hello" + ", " + name + "!"

print(greeting.upper())
print(greeting.lower())
print(len(greeting))
```

5) CONDITIONALS:

```
# conditionals.py

score = 82

if score >= 90:
    grade = "A"
elif score >= 75:
    grade = "B"
elif score >= 60:
    grade = "C"
else:
    grade = "F"

print(f"Score: {score} -> Grade: {grade}")

# Comparison and logical operators
temperature_c = 5
is_raining = True

if temperature_c < 10 and is_raining:
    print("Wear a warm, waterproof jacket.")
elif temperature_c < 10:
    print("Wear a warm jacket.")
elif is_raining:
    print("Bring an umbrella.")
else:
    print("No jacket needed.")
```

6) LOOPS:

```
# loops.py

# for loop over a range
for i in range(5):
    print(f"Count: {i}")

# for loop over a list
fruits = ["apple", "banana", "cherry"]

for fruit in fruits:
    print(f"I like {fruit}")

# while loop with a counter
countdown = 3
```

```

while countdown > 0:
    print(f"Launching in {countdown}...")
    countdown -= 1

print("Liftoff!")

# break and continue
for n in range(10):
    if n == 3:
        continue    # skip this iteration

    if n == 6:
        break        # stop the loop entirely

    print("n =", n)

```

7) FUNCTIONS:

```

# functions.py

def convert_c_to_f(celsius):
    fahrenheit = celsius * 9 / 5 + 32
    return fahrenheit

def describe_temperature(celsius, unit="C"):
    if unit == "F":
        value = convert_c_to_f(celsius)
        label = "F"
    else:
        value = celsius
        label = "C"

    return f"{value:.1f}°{label}"

print(describe_temperature(20))
print(describe_temperature(20, unit="F"))

def greet(name, times=1):
    for _ in range(times):
        print(f"Hello, {name}!")

greet("Priya")
greet("Priya", times=3)

```

8) LISTS AND STRINGS:

```
# lists_and_strings.py

students = ["Aditi", "Rahul", "Meera"]
scores = [82, 91, 76]

print(students[0], scores[0])      # indexing
print(students[-1])                # negative indexing
print(students[0:2])               # slicing

students.append("Kabir")
scores.append(88)

print(students, scores)

print("Number of students:", len(students))

# pairing list data with zip + a for loop
for name, score in zip(students, scores):
    print(f"{name}: {score}")

# basic string manipulation
report_line = "Aditi,82,Pass"

parts = report_line.split(",")
print(parts)

name, score_text, status = parts

summary = f"{name.upper()} scored {score_text} - {status}"
print(summary)

names_joined = ", ".join(students)
print("Class roster:", names_joined)
```

9) DICTIONARIES AND TUPLES:

```
# dicts_and_tuples.py

student = {
    "name": "Aditi",
    "age": 24,
    "scores": [82, 91, 76],
}

print(student["name"])
print(student.get("email", "not provided"))
```

```

student["email"] = "aditi@example.com"
student["age"] = 25

print(student)

for key, value in student.items():
    print(f"{key}: {value}")

# Tuples: fixed, ordered, immutable groupings
point = (3, 4)

x, y = point

print(f"x={x}, y={y}")

# point[0] = 10    # would raise TypeError

coordinates = [(0, 0), (3, 4), (-1, 2)]

for px, py in coordinates:
    print(f"Point: ({px}, {py})")

```

10) NESTED LOOPS AND LOGIC:

```

# nested_loops.py

# multiplication table (3x3)
for i in range(1, 4):
    for j in range(1, 4):
        print(f"{i} x {j} = {i * j}")

    print("---")

# grid search: find coordinates where a value exceeds a threshold
grid = [
    [3, 8, 2],
    [7, 1, 9],
    [4, 6, 5],
]

threshold = 6

for row_index, row in enumerate(grid):
    for col_index, value in enumerate(row):

```

```

    if value > threshold:
        print(
            f"({row_index},{col_index}) = "
            f"{value} exceeds threshold"
        )

```

11) RECURSION:

```

# recursion_basics.py

def factorial(n):
    if n <= 1:                # base case
        return 1

    return n * factorial(n - 1)  # recursive case

print(factorial(5))

def countdown(n):
    if n <= 0:
        print("Liftoff!")
        return

    print(n)
    countdown(n - 1)

countdown(3)

def sum_list(numbers):
    if not numbers:          # base case: empty list
        return 0

    return numbers[0] + sum_list(numbers[1:])

print(sum_list([4, 8, 15, 16, 23]))

```

12) MODULES AND CODE REUSABILITY:

CREATE FIRST MODULE

```

13) # helpers.py
14)
15) def is_prime(n):
16)     if n < 2:
17)         return False
18)
19)     for divisor in range(2, n):
20)         if n % divisor == 0:

```

```
21)         return False
22)
23)     return True
```

ANOTHER FILE

```
# main.py

from helpers import is_prime

print(is_prime(17))
```

CODE: helpers.py

```
# helpers.py – a module: just function definitions, nothing runs on its own

def is_prime(n):
    if n < 2:
        return False

    for divisor in range(2, n):
        if n % divisor == 0:
            return False

    return True

def is_palindrome(word):
    cleaned = word.lower()
    return cleaned == cleaned[::-1]
```

main.py

```
# main.py – imports and uses the module above

from helpers import is_prime, is_palindrome

print(is_prime(17))
print(is_palindrome("radar"))

import helpers

print(helpers.is_prime(18))
```

13) FILE I/O OPERATIONS:

```
# file_basics.py

names = ["Aditi", "Rahul", "Meera"]
```

```

with open("students.txt", "w") as f:
    for name in names:
        f.write(name + "\n")

with open("students.txt", "r") as f:
    content = f.read()
    print(content)

with open("students.txt", "r") as f:
    for line in f:
        print(f"Student: {line.strip()}")

with open("students.txt", "a") as f:
    f.write("Kabir\n")

with open("students.txt", "r") as f:
    all_lines = f.readlines()
    print(all_lines)

```

14) ERROR HANDLING: (try-except)

```

_# error_handling.py

def safe_divide(a, b):
    try:
        result = a / b

    except ZeroDivisionError:
        print("Cannot divide by zero.")
        return None

    else:
        print("Division succeeded.")
        return result

    finally:
        print("Division attempt finished.")

print(safe_divide(10, 2))
print(safe_divide(10, 0))

# Catching a specific exception type across a loop
values = ["12", "abc", "7"]

for value in values:
    try:
        number = int(value)

```

```
print(f"Converted: {number}")

except ValueError:
    print(f"'{value}' is not a valid number.")
```

15) LIBRARY: Python Standard Libraries

```
import random
from datetime import datetime

# random
dice_roll = random.randint(1, 6)
print(f"Dice roll: {dice_roll}")

colors = ["red", "green", "blue", "yellow"]
chosen_color = random.choice(colors)
print(f"Chosen color: {chosen_color}")

shuffled = colors.copy()
random.shuffle(shuffled)
print(f"Shuffled: {shuffled}")

# datetime
now = datetime.now()
print(f"Current time: {now}")
print(f"Formatted: {now.strftime('%Y-%m-%d %H:%M:%S')}")
print(f"Day of week: {now.strftime('%A')}")
```

16) OBJECT ORIENTED PROGRAMMING:

PART-A: CLASSES AND OBJECTS:

```
class Student:
    def __init__(self, name, scores):
        self.name = name
        self.scores = scores

    def average(self):
        return sum(self.scores) / len(self.scores)

    def summary(self):
        return f"{self.name}: average {self.average():.1f}"

aditi = Student("Aditi", [82, 91, 76])
rahul = Student("Rahul", [70, 65, 80])

print(aditi.summary())
print(rahul.summary())
```

```
students = [aditi, rahul]

for student in students:
    print(student.name, student.average())
```

PART-B: INHERITANCE ("is-a"):

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        return f"{self.name} makes a sound."

class Dog(Animal):
    def speak(self):
        return f"{self.name} says Woof!"

class Cat(Animal):
    def speak(self):
        return f"{self.name} says Meow!"

animals = [
    Dog("Rex"),
    Cat("Whiskers"),
    Animal("Generic Creature")
]

for animal in animals:
    print(animal.speak())

print(isinstance(Dog("Rex"), Animal))
```

CHALLENGES:

1) Challenge: save as "01C.py"

```
# hello_python.py
import sys
import platform

print("Hello, Python!")
print(
    f"You're running Python "
    f"{sys.version_info.major}."
    f"{sys.version_info.minor}."
    f"{sys.version_info.micro}"
)
print("Operating System:", platform.system())
```

2) Challenge: save as "02C.py"

```
name = "Aditi"
age = 24
weight_kg = float(input("Enter your weight in kg: "))

print(f"{name} is {age} years old and weighs {weight_kg:.1f} kg.")
```